Dominio Camping

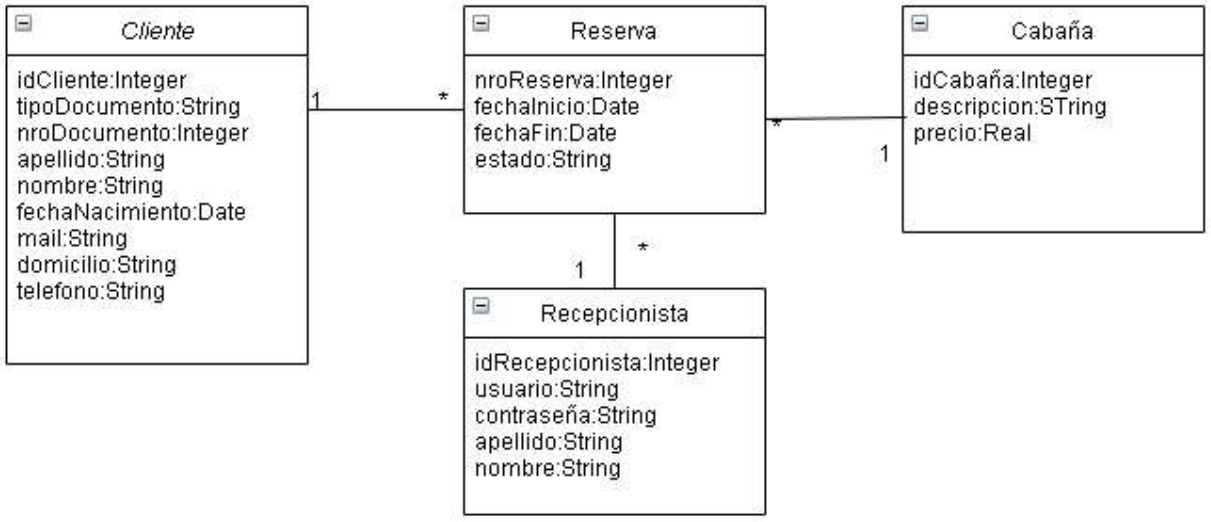
Enunciado del problema

Se desea construir un sistema para gestionar un camping. Las principales funciones del sistema serán: Gestión de cabañas, Gestión de clientes y Gestión de reservas

Reglas de negocio y especificaciones complementarias

Id.	Regla / Especificación
1	Los clientes se identifican con un id.
2	De los clientes se conoce: tipo de documento, número de documento, nombre, apellido, fecha de nacimiento, mail, domicilio y teléfono.
3	El lugar está abierto todos los días del año, inclusive los feriados.
4	Las cabañas están identificadas con un id y tienen una descripción de sus características.
5	Cada cabaña tiene un precio por día. Suponemos para simplificar que los precios no varían con el tiempo.
6	Para reservar una cabaña hay que ser cliente.
7	Un cliente puede tener varias reservas pendientes, pero no puede tener dos reservas en un mismo período.
8	No se hacen listas de espera si no hay disponibilidades.
9	Cada reserva está identificada por un número de reserva.
10	Para simplificar suponemos que una reserva se puede hacer por una sola cabaña. También suponemos que una reserva es para un único período (o sea tiene una única fecha inicio y fecha de fin de la reserva).
11	Las reservas se pueden cancelar.
12	Para verificar si una cabaña está disponible en un período se debe verificar que en ese período no tenga una reserva con estado “Vigente” o “Tomada”
13	No se puede ocupar sin previa reserva y se ocupa por el mismo periodo reservado
14	De una reserva es necesario conocer que recepcionista realizó la reserva. De los recepcionista se conoce id, usuario, contraseña, apellido y nombre
15	Las reservas pueden tener estado: Vigente (cuando se reserva), Cancelada (cuando se cancela la reserva), Tomada (cuando se realiza el check in) y Cerrada (cuando se retira el cliente)

Modelo de dominio



Caso de Uso Reservar una cabaña - Variante 2

Este Caso de Uso fue realizado basado en el dominio Camping versión 1.04 (21/05/2021)

Caso de Uso: Reservar una cabaña

Nivel de la meta: **Usuario** Alcance del Caso de Uso: **Sistema** Caja: **Negra**
Instanciación: **Real** Interacción: **Dialogal** Usabilidad: **No Contemplada**
ACTORES Primario: **Cliente** Iniciador: **Recepcionista**
PRECONDICIONES: El recepcionista esta logueado DISPARADOR: <vacio>
POSTCONDICIONES: Éxito: La reserva quedó registrada. El cliente quedo registrado.

Paso		Usuario / Condición	Sistema
1		El Recepcionista ingresa la fecha de inicio, la fecha de fin de la reserva y el precio máximo requerido	Muestra los datos de las cabañas (el id de la cabaña, el precio y la descripción de sus características), para las cabañas que estén disponibles (ver RN 12) para el período ingresado y que tengan un precio menor o igual al precio máximo requerido.
	1.a	No hay cabañas disponibles	
	1.a.1		Muestra mensaje “No hay cabañas disponibles”. Termina el CU.

Paso		Usuario / Condición	Sistema
2		El Recepcionista ingresa id_cabaña, tipo y número de documento del cliente y confirma la reserva	Valida que exista el cliente. Valida que el cliente no tenga otra reserva en ese mismo período. Registra la reserva (Asume estado “Vigente”). Muestra el número de reserva, apellido y nombre del cliente
	2.a	No existe cliente con tipo y número de documento ingresado	
	2.a.1		Muestra mensaje “No existe cliente con tipo y número de documento ingresado”. Termina el CU.
	2.b	El cliente tiene otra reserva en ese mismo período	
	2.b.1		Muestra mensaje “El cliente tiene otra reserva en ese mismo período”. Termina el CU.

Nota: Para simplificar se excluyen el resto de caminos alternativos

Interaction1

:UI

cac:ControladorAlquilerCabañas

cabañaDao:«Reposit...

cabañas[i]:Cabaña

reservas:Reserva

1: consultarDisponibilidad (fechaInicio : Date, fechaFin : Date, precioMax : Real)

1.1: disponibilidadPorPrecio (precio : Real)

cabañas = @Query 'SELECT * FROM cabañas c WHERE c.precio < precio'

«cabañas : Cabaña[*]»
1.2: disponibilidadPorPrecio

1.3: create

cabañasDisponibles:Cabaña

bucle

[,para cada c en cabañas]

1: consultarDisponibilidad

disponible = true

bucle

[,para cada reserva en reservas]

1: estaDisponible

si fechaInicio esta entre (reserva.fechaInicio y reserva.fechaFin) o
fecha fin esta entre (reserva.fechaInicio y reserva.fechaFin)
ocupada = true
sino
ocupada = false

«ocupada»
2: estaDisponible

si ocupada = true
disponible = false
cortar bucle

«disponible»
2: consultarDisponibilidad

opcional

[si disponible = true]

1: add

2: add

«cabañasDisponibles : Cabaña [*]»
2: consultarDisponibilidad

alternativo

[cabañasDisponibles IS EMPTY]

UI muestra al recepcionista mensaje: "No hay cabañas disponibles"

[cabañasDisponibles IS NOT EMPTY]

bucle

[,para cada cabaña en cabañasDisponibles]

1: getId

«idCabaña»
2: getId

3: getPrecio

«precio»
4: getPrecio

5: getDescripcion

«descripcion»
6: getDescripcion

UI muestra al recepcionista: { idCabaña + precio + descripcion }

:UI
cac:ControladorAlquilerCabañas
clienteDao:«Repository»ClienteDao
clienteActual:Cliente
reservasCliente:Reserva
cabañaDao:«Repository»CabañaDao
reservaDao:«Repository»ReservaDao

1: confirmarReserva (idCabaña : Integer, tipoDoc : String, nroDoc : String)

MensajeError = ''

1.1: getOne (tipoDoc : String, nroDoc : String)

«clienteActual : Cliente»
1.2: getOne

alternativo

[clienteActual IS NULL]

MensajeError = 'No existe cliente con tipo y nro documento ingresado'

[clienteActual IS NOT NULL]

1: disponibilidadEntreFechas

disponible = true

bucle

[,para cada r en reservas]

1: fechasClienteDisponibles

si r.estado = vigente y
(fechaInicio esta entre (r.fechaInicio y reserva.fechaFin) o
fecha fin esta entre (r.fechaInicio y reserva.fechaFin))
  periodoOcupado = true
sino
  periodoOcupado = false

«periodoOcupado»
2: fechasClienteDisponibles

si periodoOcupado = true
  disponible = false
  cortar bucle

«disponible»
2: disponibilidadEntreFechas

alternativo

[si disponible = false]

MensajeError = 'El cliente tiene otra reserva en ese mismo período'

[si disponible = true]

1: getOne

«cabañaActual : Cabaña»
2: getOne

3: create

nuevaReserva:Reserva

4: inicializar

se genera un numero de reserva y se define el estado = 'Vigente'

5: inicializar

6: add

7: add

«clienteActual : Cliente, reservaActual : Reserva, MensajeError»
2: confirmarReserva

alternativo

[MensajeError IS NOT NULL]

UI muestra al actor: MensajeError

[MensajeError IS NULL]

1: getNroReserva

«nroReserva»
2: getNroReserva

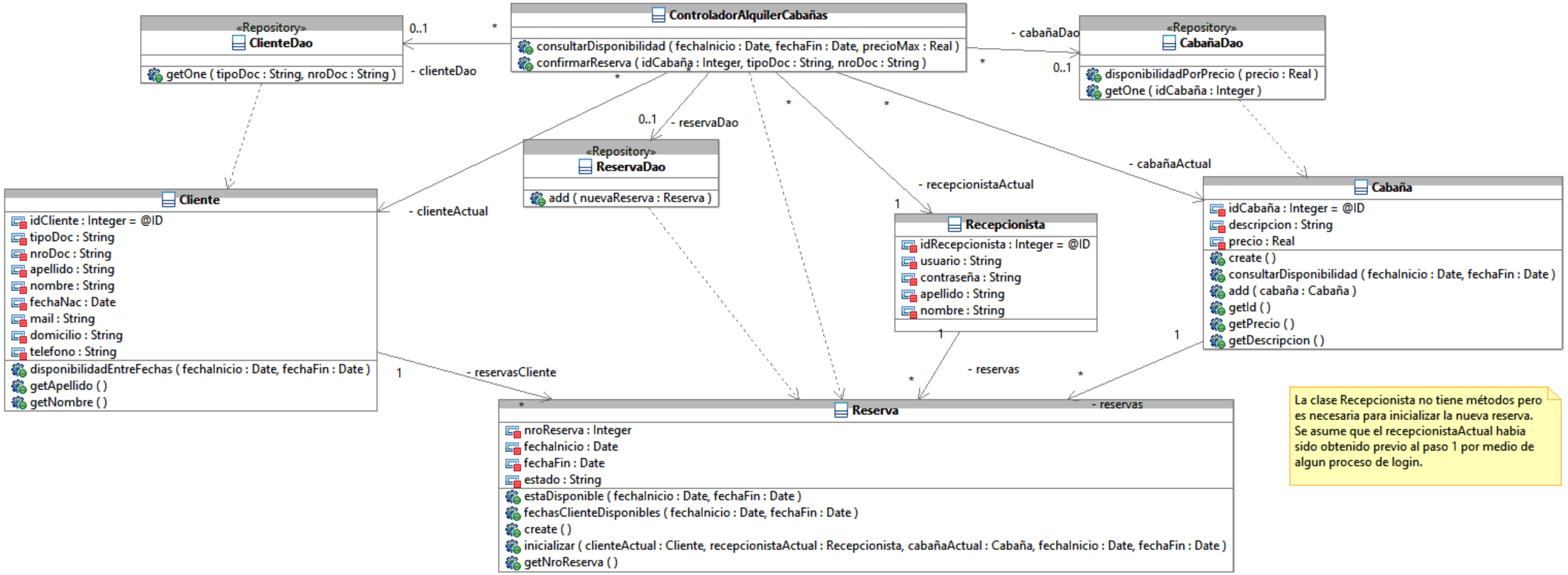
3: getApellido

«apellido»
4: getApellido

5: getNombre

«nombre»
6: getNombre

UI muestra mensaje: nroReserva + apellido + nombre


«Repository»
ClienteDao
getOne (tipoDoc : String, nroDoc : String)
0..1
*
- clienteDao
ControladorAlquilerCabañas
consultarDisponibilidad (fechaInicio : Date, fechaFin : Date, precioMax : Real)
confirmarReserva (idCabaña : Integer, tipoDoc : String, nroDoc : String)
- cabañaDao
«Repository»
CabañaDao
disponibilidadPorPrecio (precio : Real)
getOne (idCabaña : Integer)
0..1
«Repository»
ReservaDao
add (nuevaReserva : Reserva)
0..1
- reservaDao
Cliente
idCliente : Integer = @ID
tipoDoc : String
nroDoc : String
apellido : String
nombre : String
fechaNac : Date
mail : String
domicilio : String
telefono : String
disponibilidadEntreFechas (fechaInicio : Date, fechaFin : Date)
getApellido ( )
getNombre ( )
- clienteActual
- recepcionistaActual
- cabañaActual
Recepcionista
idRecepcionista : Integer = @ID
usuario : String
contraseña : String
apellido : String
nombre : String
Cabaña
idCabaña : Integer = @ID
descripcion : String
precio : Real
create ( )
consultarDisponibilidad (fechaInicio : Date, fechaFin : Date)
add (cabaña : Cabaña)
getId ( )
getPrecio ( )
getDescripcion ( )
1
- reservasCliente
- reservas
- reservas
Reserva
nroReserva : Integer
fechaInicio : Date
fechaFin : Date
estado : String
estaDisponible (fechaInicio : Date, fechaFin : Date)
fechasClienteDisponibles (fechaInicio : Date, fechaFin : Date)
create ( )
inicializar (clienteActual : Cliente, recepcionistaActual : Recepcionista, cabañaActual : Cabaña, fechaInicio : Date, fechaFin : Date)
getNroReserva ( )
La clase Recepcionista no tiene métodos pero es necesaria para inicializar la nueva reserva. Se asume que el recepcionistaActual habia sido obtenido previo al paso 1 por medio de algun proceso de login.